

CM0081 Formal Languages and Automata

Introduction to AGDA

Andrés Sicard-Ramírez

Universidad EAFIT

Semester 2024-1

(Last modification: 4th November 2025)

Propositions-as-Types Principle

Three correspondence's levels

Wadler (2015) introduces correspondence's levels by:

(i) Propositions-as-types

For each proposition in the logic there is a corresponding type in the programming language—and vice versa.

Propositions-as-Types Principle

Three correspondence's levels

Wadler (2015) introduces correspondence's levels by:

(i) Propositions-as-types

For each proposition in the logic there is a corresponding type in the programming language—and vice versa.

(ii) Proofs-as-programs

For each proof of a given proposition, there is a program of the corresponding type—and vice versa.

Propositions-as-Types Principle

Three correspondence's levels

Wadler (2015) introduces correspondence's levels by:

(i) Propositions-as-types

For each proposition in the logic there is a corresponding type in the programming language—and vice versa.

(ii) Proofs-as-programs

For each proof of a given proposition, there is a program of the corresponding type—and vice versa.

(iii) Simplification of proofs as evaluation of programs

For each way to simplify a proof there is a corresponding way to evaluate a program—and vice versa.

Propositions-as-Types Principle

Other names

- The Curry-Howard correspondence/isomorphism

[†](Sørensen and Urzyczyn 2006, p. viii).

Propositions-as-Types Principle

Other names

- The Curry-Howard correspondence/isomorphism
- The Brouwer - Heyting - Kolmogorov - Schönfinkel - Curry - Meredith - Kleene - Feys - Gödel - Läuchli - Kreisel - Tait - Lawvere - Howard - de Bruijn - Scott - Martin-Löf - Girard - Reynolds - Stenlund - Constable - Coquand - Huet - ... - correspondence[†]

[†](Sørensen and Urzyczyn 2006, p. viii).

Natural Deduction (Conjunction and Implication)

Preliminaries

- Propositions: $A, B, C, \dots$
- Judgement: A true (assert, proposition A is true)
- Form of the inference rules

$$\frac{J_1 \quad \dots \quad J_n}{J} \text{ rule name}$$

where J (conclusion) and $J_1, \dots, J_n$ (premises) are all judgements.

- Types of inference rules: Introduction rules and elimination rules

Natural Deduction (Conjunction and Implication)

Inference rules for conjunction

- Introduction rule (composing information)

$$\frac{A \text{ true} \quad B \text{ true}}{A \wedge B \text{ true}} \wedge \text{I}$$

- Elimination rules (retrieving/using information)

$$\frac{A \wedge B \text{ true}}{A \text{ true}} \wedge \text{E}_1$$

$$\frac{A \wedge B \text{ true}}{B \text{ true}} \wedge \text{E}_2$$

Natural Deduction (Conjunction and Implication)

Inference rules for implication

- Introduction rule (hypothetical judgement)

$$\frac{\begin{array}{c} [A \text{ true}]^i \\ \vdots \\ B \text{ true} \end{array}}{A \supset B \text{ true}} \supset I^i$$

- Elimination rule (modus ponens)

$$\frac{A \supset B \text{ true} \quad A \text{ true}}{B \text{ true}} \supset E$$

Natural Deduction (Conjunction and Implication)

Example

A proof that $(A \wedge B) \supset (B \wedge A)$ true.

$$\frac{\frac{[A \wedge B \text{ true}]^i}{B \text{ true}} \wedge E_2 \quad \frac{[A \wedge B \text{ true}]^i}{A \text{ true}} \wedge E_1}{B \wedge A \text{ true}} \wedge I$$
$$\frac{B \wedge A \text{ true}}{(A \wedge B) \supset (B \wedge A) \text{ true}} \supset I^i$$

Typed Lambda Calculus (Product and Function Types)

- Types

$$\begin{array}{l} \sigma, \tau ::= \sigma \rightarrow \tau \\ \quad | \sigma \times \tau \end{array}$$

function type

product type

- λ -terms

$$\begin{array}{l} M, N ::= x \\ \quad | \lambda x. M \\ \quad | M N \\ \quad | \langle M, N \rangle \mid \text{fst } M \mid \text{snd } M \end{array}$$

variable

λ -abstraction

application

pairs and projections

- Judgement: $M : \sigma$ (λ -term M has type σ)

Typed Lambda Calculus (Product and Function Types)

Type assignment rules for product types

- Introduction rule (pair formation)

$$\frac{M : \sigma \quad N : \tau}{\langle M, N \rangle : \sigma \times \tau} \times\text{I}$$

- Elimination rules (pair projections)

$$\frac{M : \sigma \times \tau}{\text{fst } M : \sigma} \times\text{E}_1$$

$$\frac{M : \sigma \times \tau}{\text{snd } M : \tau} \times\text{E}_2$$

Typed Lambda Calculus (Product and Function Types)

Type assignment rules for function types

- Introduction rule (λ -abstraction)

$$\frac{\begin{array}{c} [x : \sigma]^i \\ \vdots \\ M : \tau \end{array}}{\lambda x. M : \sigma \rightarrow \tau} \rightarrow I^i$$

- Elimination rule (application)

$$\frac{M : \sigma \rightarrow \tau \quad N : \sigma}{M N : \tau} \rightarrow E$$

Typed Lambda Calculus (Product and Function Types)

Example

A proof that $\lambda h. \langle \text{snd } h, \text{fst } h \rangle : (\sigma \times \tau) \rightarrow (\tau \times \sigma)$.

$$\frac{\frac{[h : \sigma \times \tau]^i}{\text{snd } h : \tau} \times E_2 \quad \frac{[h : \sigma \times \tau]^i}{\text{fst } h : \sigma} \times E_1}{\langle \text{snd } h, \text{fst } h \rangle : \tau \times \sigma} \times I \quad \rightarrow I^i$$

Correspondence's Levels

Example (propositions as types)

(implication) $A \supset B$ as $\sigma \rightarrow \tau$ (function type)

(conjunction) $A \wedge B$ as $\sigma \times \tau$ (product type)

Correspondence's Levels

Example (proofs as programs)

$$\frac{\frac{[A \wedge B \text{true}]^i}{B \text{true}} \wedge E_2 \quad \frac{[A \wedge B \text{true}]^i}{A \text{true}} \wedge E_1}{B \wedge A \text{true}} \wedge I$$
$$\frac{B \wedge A \text{true}}{(A \wedge B) \supset (B \wedge A) \text{true}} \supset I^i$$

Proof

$$\frac{\frac{[h : \sigma \times \tau]^i}{\text{snd } h : \tau} \times E_2 \quad \frac{[h : \sigma \times \tau]^i}{\text{fst } h : \sigma} \times E_1}{\langle \text{snd } h, \text{fst } h \rangle : \tau \times \sigma} \times I$$
$$\frac{\langle \text{snd } h, \text{fst } h \rangle : \tau \times \sigma}{\lambda h. \langle \text{snd } h, \text{fst } h \rangle : (\sigma \times \tau) \rightarrow (\tau \times \sigma)} \rightarrow I^i$$

Program

Proof Assistants

Description

*“Proof assistants are computer systems that allow a user to do mathematics on a computer, but not so much the computing (numerical or symbolical) aspect of mathematics but the aspects of **proving** and **defining**. So a user can **set up** a mathematical theory, define properties and do logical reasoning with them.”*
(Geuvers 2009, p. 3)

Example

- Based on set theory: ISABELLE/ZFC, METAMATH and MIZAR
- Based on higher-order logic: HOL4, HOL LIGHT and ISABELLE/HOL
- Bases on type theories: AGDA, COQ and LEAN.

What is AGDA?

- Dependently typed functional programming language
- Dependently typed interactive proof assistant

Long tradition: The ALF/AGDA family (Gothenburg - Sweden)

- ALF
- AGDA
- ALFA. Graphical interface for AGDA.
- AGDALIGHT. Experimental version of AGDA.
- AGDA 2
 - Based on **Martin-Löf Type Theory** (also known as **Constructive Type Theory** or **Intuitionistic Type Theory**).
 - Direct manipulation of proofs-objects.
 - Backends to `HASKELL` and `JAVASCRIPT`.
 - Written in `HASKELL`.
 - Interaction via `EMACS`.

Further Reading

Propositions-as-types principle

- Wadler (2015). Propositions as Types.
- Sørensen and Urzyczyn (2006). Lectures on the Curry-Howard Isomorphism.

AGDA

- Bove and Dybjer (2009). Dependent Types at Work.
- Norell (2009). Dependently Typed Programming in Agda.
- Stump (2016). Verified Functional Programming in Agda.
- Altenkirch (2024). The Tao of Types.

Demo

References

- Thorsten Altenkirch (2024). The Tao of Types. (In preparation) (cit. on p. 20).
- Ana Bove and Peter Dybjer (2009). Dependent Types at Work. In: LerNet ALFA Summer School 2008. Ed. by Ana Bove, Luís Soares Barbosa, Alberto Pardo and Jorge Sousa Pinto. Vol. 5520. Lecture Notes in Computer Science. Springer, pp. 57–99. DOI: [10.1007/978-3-642-03153-3_2](https://doi.org/10.1007/978-3-642-03153-3_2) (cit. on p. 20).
- H. Geuvers (2009). Proof Assistants: History, Ideas and Future. *Sadhana* 34.1, pp. 3–25. DOI: [10.1007/s12046-009-0001-5](https://doi.org/10.1007/s12046-009-0001-5) (cit. on p. 17).
- Ulf Norell (2009). Dependently Typed Programming in Agda. In: Advanced Functional Programming (AFP 2008). Ed. by Pieter Koopman, Rinus Plasmeijer and Doaitse Swierstra. Vol. 5832. Lecture Notes in Computer Science. Springer, pp. 230–266. DOI: [10.1007/978-3-642-04652-0_5](https://doi.org/10.1007/978-3-642-04652-0_5) (cit. on p. 20).
- Morten-Heine Sørensen and Paul Urzyczyn (2006). Lectures on the Curry-Howard Isomorphism. Vol. 149. Studies in Logic and the Foundations of Mathematics. Elsevier (cit. on pp. 5, 6, 20).
- Aaron Stump (2016). Verified Functional Programming in Agda. ACM and Morgan & Claypool. DOI: [10.1145/2841316](https://doi.org/10.1145/2841316) (cit. on p. 20).
- Philip Wadler (2015). Propositions as Types. *Communications of the ACM* 58.12, pp. 75–84. DOI: [10.1145/2699407](https://doi.org/10.1145/2699407) (cit. on pp. 2–4, 20).